

Documentation de l'assembleur du processeur PIso

Introduction

asm-PIso est un assembleur pour le processeur PIso, processeur pédagogique dédié à la compréhension et à l'apprentissage.

Ce processeur est réalisé sous [Logisim](#) ([Sources](#)).

Structure d'un programme

Un programme est un fichier texte composé de lignes contenant :

- soit un commentaire,
- soit une directive,
- soit une déclaration d'une donnée,
- soit une instruction,
- soit une ligne vide.

Un fichier programme doit comporter l'extension **.piso** et le fichier exécutable produit portera le même nom avec l'extension **.exe**.
Chaque ligne se termine par un caractère **newline** ('\n') et le programme se termine par une ligne vide.

Un programme est composé de 2 parties, d'abord la zone données et ensuite la zone programme.

Zone données

La zone données est la partie du programme où sont déclarées les variables. Elle commence par la directive **.data**. Elle contient toutes les variables du programme, qui doivent donc toutes être déclarées dans cette zone.

Zone programme

La zone programme est la partie qui contient le programme proprement dit, c'est à dire la suite d'instructions qui le constitue. Elle commence par la directive **.text**.

Syntaxe

Séparateur

Tous les mots du langage doivent être séparés par des espaces.

Commentaires

Les commentaires sont des lignes entières qui commencent par le caractère **#**

Exemple

```
#  
# Voici un petit programme qui détermine le maximum entre deux variables  
#
```

Lors de l'assemblage d'un programme, les commentaires rencontrés dans le fichier source seront intégralement reproduits dans le fichier exécutable sous forme de commentaires.

Des commentaires seront également ajoutés par l'assembleur, ceux-ci commenceront par la séquence **# -**

Constantes

Une constante est une valeur sur un octet, exprimée en hexadécimal sur deux chiffres. Elle se trouve dans des instructions ou en initialisation lors de la déclaration de variables. Les chiffres hexadécimaux de A à F peuvent s'écrire en majuscule ou en minuscule.

Exemple

```
#  
# Voici une constante dans une instruction d'affectation de variable,  
# la variable se nomme PI et la constante vaut 3B  
#
```

```
PI := 3B
```

Variables

Une variable représente une adresse. Elle commence par un caractère alphabétique différent des chiffres hexadécimaux, c'est à dire de 'g' (ou 'G') à 'z' (ou 'Z') et se continue par des caractères alphanumériques ou le caractère *tiret bas* '_'.

Une variable doit être déclarée une et une seule fois dans la zone **.data**.

Une variable peut se trouver autant de fois que nécessaire en argument d'une instruction.

Exemple

```
#
# Voici des variables dans des instructions les variables sont respectivement : X, Somme, p_22bis
#
```

```
X := 23
Somme := Somme + p_22bis
```

Étiquettes

Une étiquette représente une adresse dans le code. Elle commence par un caractère alphabétique, se continue par des caractères alphanumériques ou le caractère *tiret bas* '_' et se termine par le caractère *deux points* ':'.
Une étiquette doit être placée une et une seule fois en début de ligne dans le code, cette ligne permettant de l'initialiser avec l'adresse courante dans le code.
Une étiquette peut se trouver autant de fois que nécessaire en argument d'une instruction de branchement où elle représente l'adresse de destination du branchement.

Exemple

```
#
# Voici une boucle sans fin
#
```

Boucle: VERS Boucle:

Instructions

Une instruction tient sur une seule ligne et se termine par un caractère **newline** ('\n').

Exemple

```
#
# Voici une instruction de branchement inconditionnel à l'adresse LA_BAS
#
```

VERS LA_BAS:

Déclaration des variables

Les variables doivent être déclarées en début de programme, dans la zone **.data** Une variable est déclarée sur une ligne, en indiquant dans l'ordre son nom puis son adresse et éventuellement sa valeur initiale. Une variable sans valeur initiale sera initialisée à 0.

Exemples

```
#
# Déclaration de la variable P à l'adresse 0100 initialisée à la valeur 0
#
P 100

#
# Déclaration de la variable TOTO à l'adresse 1234 initialisée à la valeur 127
#
TOTO 1234 7f

#
# Déclaration de la variable x_92 à l'adresse 51966 initialisée à la valeur -17
#
x_92 CAFE EF
```

Instructions et opérations

Le langage possède 7 instructions, 4 opérations arithmétiques et 6 opérations logiques

Opérations arithmétiques

Les 4 opérations arithmétiques sont l'addition '+', la soustraction '-', la multiplication '*' et la division '/'. Elles opèrent sur des entiers signés sauf la division qui opère sur des entiers non signés.

Opérations logiques

Les 6 opérations logiques sont les opérations de comparaison : '=', '<>', '>', '<', '>=', '<='. Ils comparent des variables ou des constantes entières signées.

Instructions

Les instructions se trouvent dans la zone **.text** du programme. Le langage possède 7 instructions qui se combinent avec les opérateurs (ou pas).
Dans la suite :

- **X**, **Y** ou **Z** représentent des variables,
- **CONST** représente une constante,

- **opa** représente un opérateur arithmétique,
- **opl** représente un opérateur logique
- **ET** représente une étiquette

X := CONST

Range la valeur de la constante **cte** dans la case d'adresse **X**.

X := Y

Range la valeur contenue à l'adresse **Y** dans la case d'adresse **X**.

X := Y opa CONST

Range le résultat du calcul **Y opa cte** dans la case d'adresse **X**.

X := Y opa Z

Range le résultat du calcul **Y opa Z** dans la case d'adresse **X**.

Vers ET

Déroute le programme à l'adresse **ET**, c'est à dire que la prochaine instruction à exécuter est celle à l'adresse **ET**. Attention à respecter la casse.

Si X opl CONST Vers ET

Si le résultat du calcul **X opl cte** est vrai, l'instruction déroute le programme à l'adresse **ET**, sinon le programme se continue à l'instruction suivante. Attention à respecter la casse.

Si X opl Y Vers ET

Si le résultat du calcul **X opl Y** est vrai, l'instruction déroute le programme à l'adresse **ET**, sinon le programme se continue à l'instruction suivante. Attention à respecter la casse.

Exemple de programme

Le programme suivant calcule la somme des N premiers nombres pairs :

```
#
# Exemple de programme
# Ce programme calcule la somme des N premiers nombres pairs
#
# Zone des variables
#
.data
# Nombre d'entiers pairs a additionner
N 0100 a
# Valeur paire a accumuler la 1re est 2
VP 0101 2
# Comme son nom l'indique ! Initialisée a 0
SOMME 0102
#
# Zone du code
#
.text
DEBUT: Si N <= 0 Vers FIN:
SOMME := SOMME + VP
N := N - 1
VP := VP + 2
Vers DEBUT:
FIN: Vers FIN:
```